LAB ASSIGNMENT - 8.2

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task 1 – Test-Driven Development for Even/Odd Number Validator

• Use AI tools to first generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests.

Requirements:

• Input must be an integer

• Handle zero, negative numbers, and large integers

Example Test Scenarios:

is_even(2) → True

is_even(7) → False

is_even(0) → True

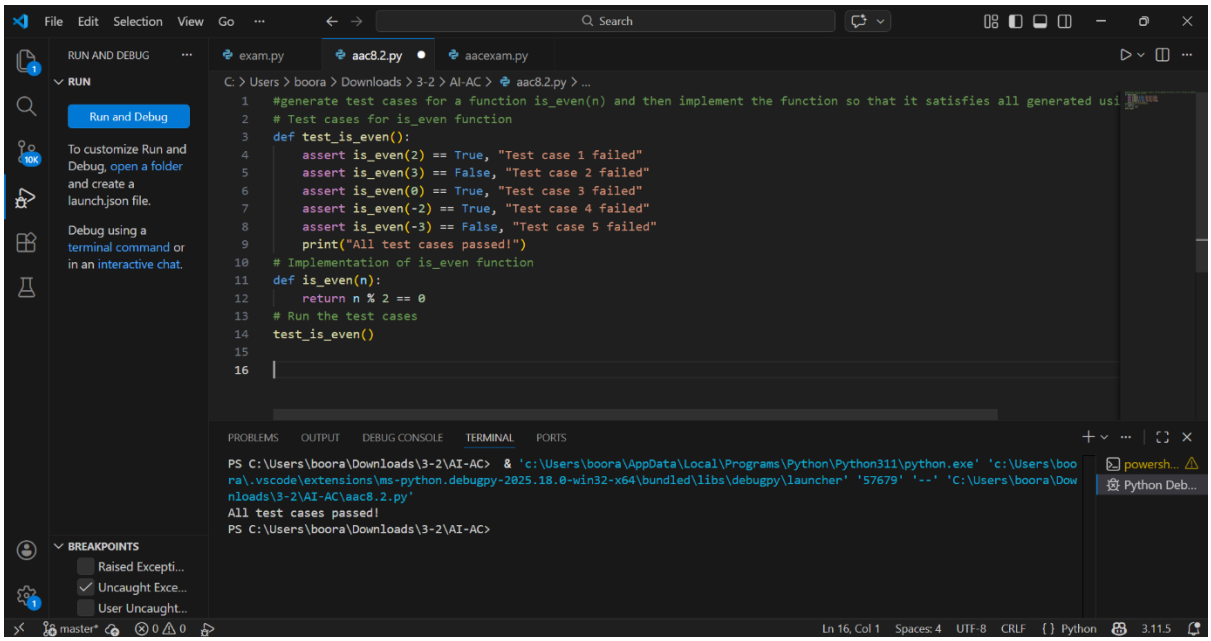
is_even(-4) → True

is_even(9) → False

Expected Output -1

• A correctly implemented is_even() function that passes all AI-generated test cases

Code & Output:



Justification:

This task demonstrates Test-Driven Development by writing test cases before implementing the function.
It ensures correctness for integers including zero, negative, and large values.
The validator improves reliability of numeric validation logic in real software systems.

Task 2 – Test-Driven Development for String Case Converter

• Ask AI to generate test cases for two functions:

• to_uppercase(text)

• to_lowercase(text)

Requirements:

• Handle empty strings

• Handle mixed-case input

• Handle invalid inputs such as numbers or None

Example Test Scenarios:

to_uppercase("ai coding") → "AI CODING"

to_lowercase("TEST") → "test"

to_uppercase("") → ""

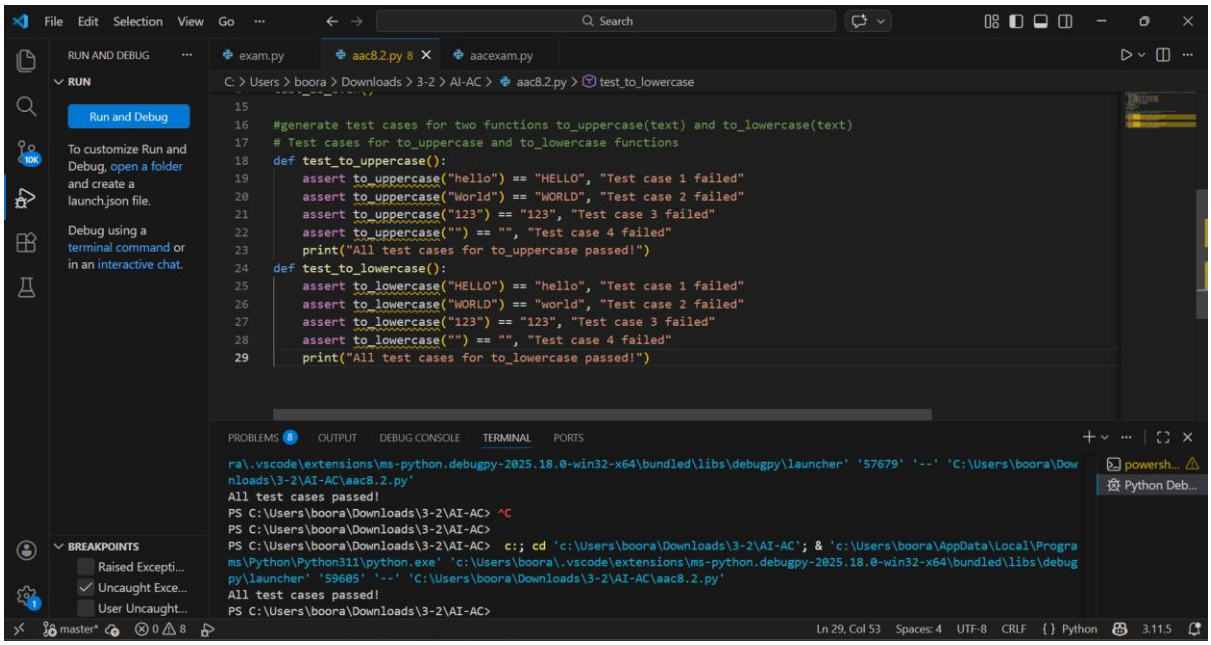
to_lowercase(None) → Error or safe handling

Expected Output -2

• Two string conversion functions that pass all AI-generated test

cases with safe input handling.

Code & Output:



Justification:

This task applies TDD to verify safe string manipulation functions.
It handles edge cases such as empty strings, mixed case text, and invalid inputs.
The implementation ensures robust text processing used in user input handling systems.

Task 3 – Test-Driven Development for List Sum Calculator

• Use AI to generate test cases for a function sum_list(numbers)

that calculates the sum of list elements.

Requirements:

• Handle empty lists

• Handle negative numbers

• Ignore or safely handle non-numeric values

Example Test Scenarios:

sum_list([1, 2, 3]) → 6

sum_list([]) → 0

sum_list([-1, 5, -4]) → 0

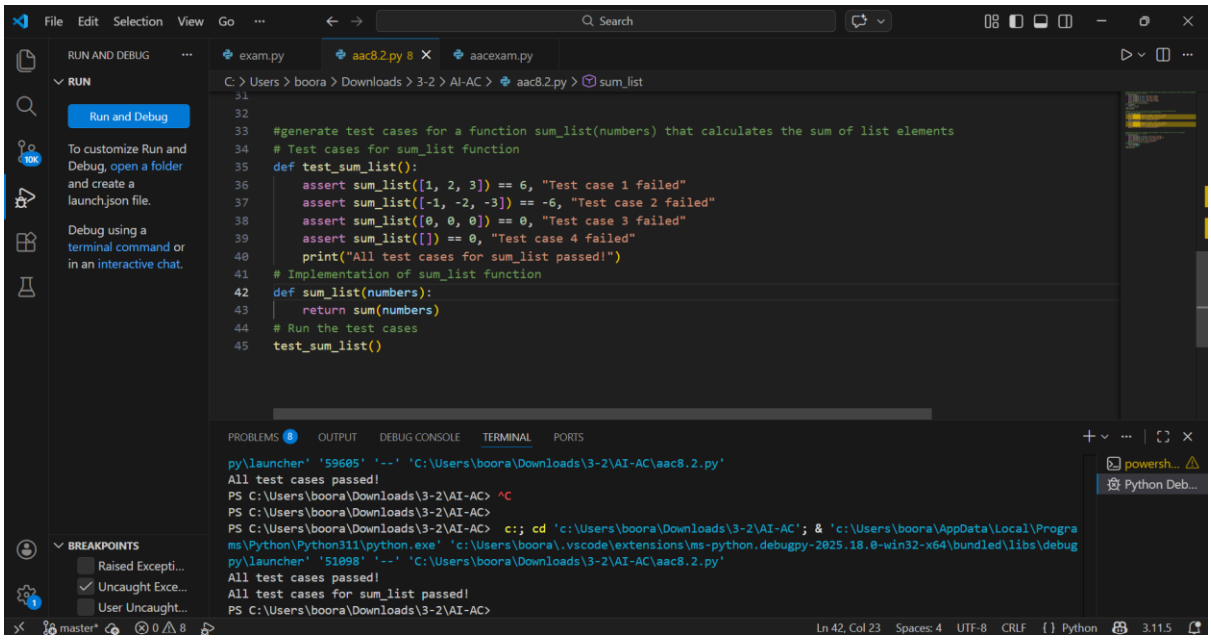
sum_list([2, "a", 3]) → 5

Expected Output 3

• A robust list-sum function validated using AI-generated test

cases.

Code & Output:



Justification:

This task validates summation logic using AI-generated test cases.
It ensures proper handling of empty lists, negative numbers, and non-numeric elements.
The function improves data processing reliability in real-world applications.

Task 4 – Test Cases for Student Result Class

• Generate test cases for a StudentResult class with the following

methods:

• add_marks(mark)

• calculate_average()

• get_result()

Requirements:

• Marks must be between 0 and 100

• Average ≥ 40 → Pass, otherwise Fail

Example Test Scenarios:

Marks: [60, 70, 80] → Average: 70 → Result: Pass

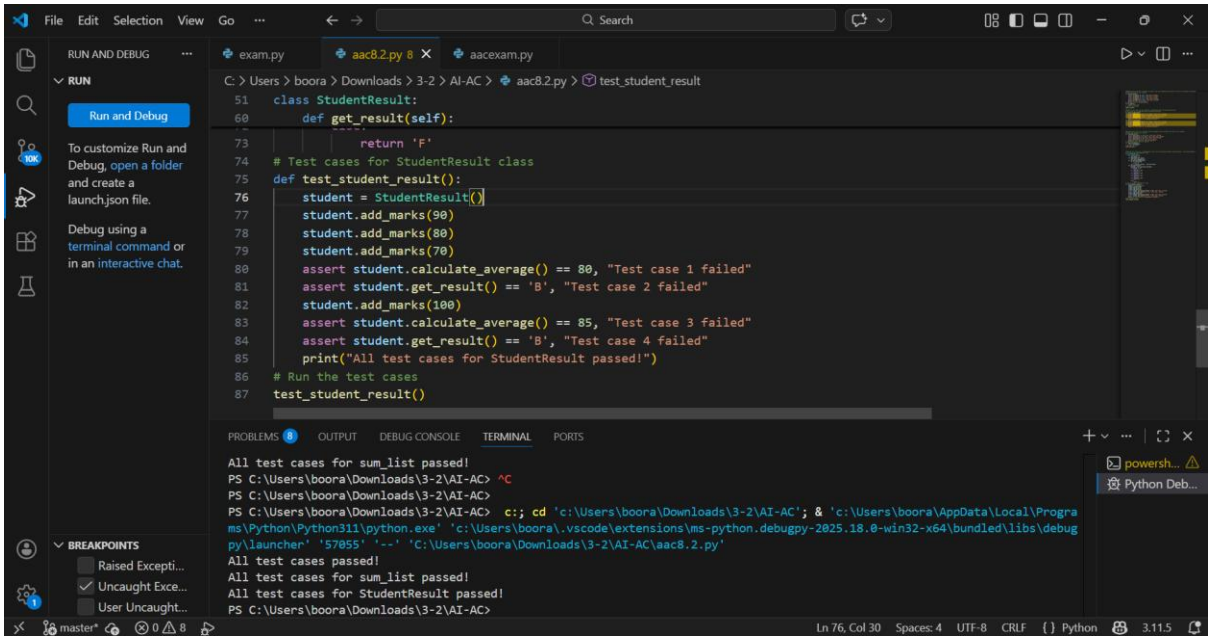
Marks: [30, 35, 40] → Average: 35 → Result: Fail

Marks: [-10] → Error

Expected Output -4

• A fully functional StudentResult class that passes all AI-generated test

Code & Output:



Justification:

This task tests an object-oriented grading system using predefined constraints.
It verifies mark validation, average calculation, and pass/fail decision logic.
The class models real academic evaluation systems with safe input validation.

Task 5 – Test-Driven Development for Username Validator

Requirements:

• Minimum length: 5 characters

• No spaces allowed

• Only alphanumeric characters

Example Test Scenarios:

is_valid_username("user01") → True

is_valid_username("ai") → False

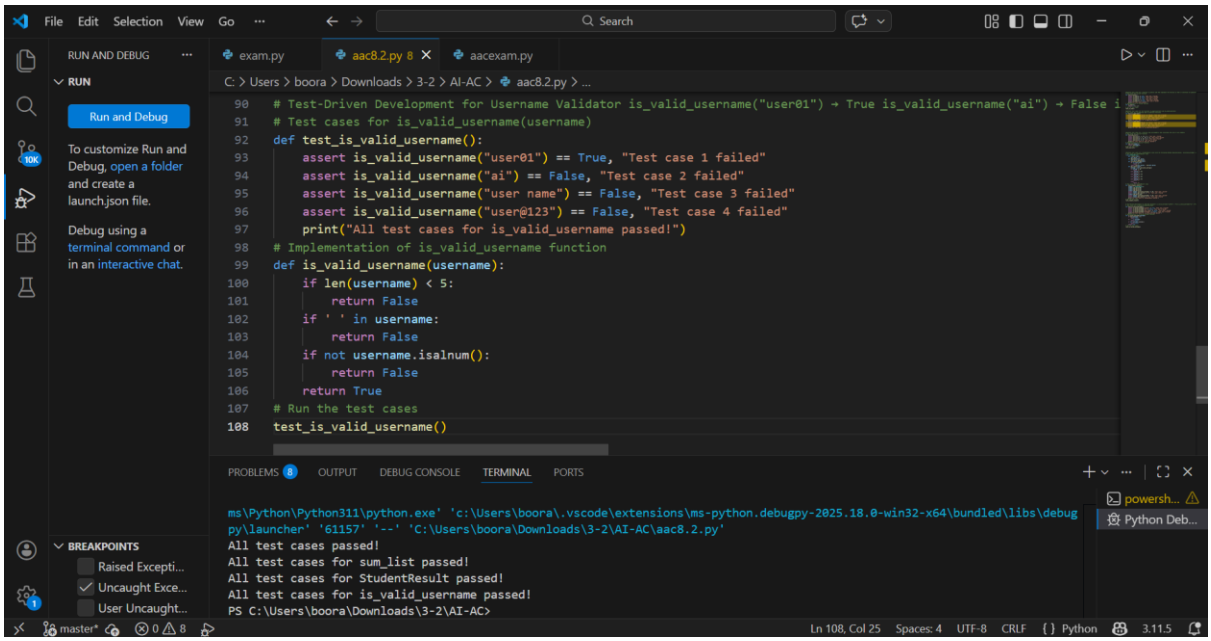
is_valid_username("user name") → False

is_valid_username("user@123") → False

Expected Output 5

A username validation function that passes all AI-generated test cases.

Code & Output:



Justification:

This task ensures secure username validation through rule-based testing.
It prevents invalid usernames such as short, spaced, or special-character inputs.
The validator improves authentication system reliability and user data integrity.